

Final Assignment: Design Note and Evaluation Set

Doc-to-Podcast Pipeline (Option 3)

Author: Aafaqueahmad Khan

Part 1: Design Note

Which Architecture Pattern I Chose, and Why

This is my first generative AI project, so I first asked the model to create the whole script in one call. The result was short and felt more like a summary than an episode. So I used a pattern called plan and execute. The model first creates an outline, and then each section gets its own call using only the relevant part of the document. A second call checks each completed section against the document. I learned that when a model is asked to check its own work in the same call, it often says the work is fine.

One Failure I Hit, and How I Debugged It

The last step saves the episode using a tool on an MCP server. At first, the tool took the transcript text directly, so the model had to include the whole script in its tool call. It failed on every run with an unterminated string error. I first thought there was a problem in my code. The actual cause was the model's output limit. My limit was 300 tokens, so the script was cut off halfway through. Increasing the limit would only cause the same problem with a longer episode. I changed the tool so that it takes a file path instead. The script is now saved to a file, and the tool reads the file directly.

One Tradeoff I Made

My document is small and has only six pieces, so I could send the complete document every time or search for the pieces I need. I decided to use both approaches. The outline and checking steps use the complete document because a checker that sees only a few pieces may mark a true fact as wrong if the supporting information is missing. Section writing uses search and selects the best four pieces. This can result in a weaker section if the search misses important information, but it gives more focused and specific writing.

Part 2: Evaluation Set

My tests are in `eval_set.py`, and `eval_runner.py` runs them. Nothing is mocked. The runner creates a real episode from the real document using the real model and the real save tool. It then checks the result.

I wrote ten tests in three groups. The first four are questions I would ask someone who had listened to the episode, and I know the answers from the document. The next three check the pipeline itself. The last three test the checking step directly using text that I wrote myself. I added these tests because a normal run usually has no incorrect information, so the checker may find nothing. These tests show whether it can actually find mistakes.

I ran the tests with this command:

```
python eval_runner.py
```

Result: 10 out of 10 passed.

The run created an episode called "Building a Centralized Design System: A Technical Case Study." It was 235 seconds of audio, used 26,197 tokens, and took about 44 seconds.

Group A: Four Questions About the Finished Episode

Test Case - 1.

Question: Which frontend framework is the design system built on?

Expected answer: Angular, using Angular Material.

Result: PASS. I look for both names because mentioning only Angular does not include the component library that the document specifically mentions.

Test Case - 2.

Question: How long did the project take compared with the original plan?

Expected answer: About six months, compared with a twelve month plan.

Result: PASS. I look for both numbers because saying only six months removes the comparison that gives the number its meaning.

Test Case - 3.

Question: How is the design system documented and shared with other teams?

Expected answer: It is documented in Storybook and shared as an npm package.

Result: PASS. Both are required because the document describes both. An episode that mentions only one has answered only part of the question.

Test Case - 4.

Question: Does the episode name any framework that the document does not mention?

Expected answer: No. The document only mentions Angular.

Result: PASS. React, Vue, and Svelte did not appear. This is the only test that checks whether the model adds information that is not in the document.

Group B: Three Checks on the Pipeline

Test Case - 5.

Input: The outline created by the model.

Expected: Between three and seven sections. Fewer than three means the model combined too much information. More than seven means it divided the document into sections that are too small to read.

Result: PASS. It created six sections.

Test Case - 6.

Input: The script and the audio file created from it.

Expected: At least one second of audio for every five words in the script.

Result: PASS. This is measured against the script instead of using a fixed number of seconds. It can therefore detect audio that stops early even when the episode is long.

Test Case - 7.

Input: The two files reported by the saving step.

Expected: Both files exist on disk, the audio file is not empty, and the saved text is exactly the same as the generated text.

Result: PASS. Comparing the two texts is important. Checking only that the file exists could still pass if the file was incomplete. This was similar to the problem caused by my original truncation bug.

Group C: Three Tests on the Checking Step

Test Case - 8.

Input: A paragraph saying the project took eighteen months and used React, Vue, and Svelte. None of these details are in the document.

Expected: The critic flags these claims.

Result: PASS. It caught both the eighteen month claim and the frameworks.

Test Case - 9.

Input: A paragraph where everything is correct except one number. It says the team finished in seven months instead of six, while the rest of the sentence closely matches the document.

Expected: The critic flags the seven month claim.

Result: PASS. This was the test I expected to fail because the mistake is small and the rest of the information is correct.

Test Case - 10.

Input: A paragraph saying the team finished in about half the planned time using Angular. This is true, but it is written differently on purpose.

Expected: The critic flags nothing.

Result: PASS. It flagged nothing.